

any sort of microprocessor. That processor may be, for instance, the embedded soft processor in an FPGA, a hard ARM processor or an external x86 processor, as depicted in Fig. 2.

At a certain point during the execution of this host software routine, there is likely to be a function that is computationally expensive and can benefit from the highly parallel acceleration on a more parallel device like a CPU, GPU or FPGA. This function to be accelerated is referred to as an OpenCL kernel.

These kernels are written in standard 'C' language, but annotated with constructs to specify parallelism and memory hierarchy. The example shown in Fig. 3 performs vector addition of two arrays, 'a' and 'b', while writing the results back to an output array answer. Parallel threads operate on each element of the vector, allowing the result to be computed much more quickly when it is accelerated by a device that offers massive amounts of fine-grained parallelism, such as an FPGA. The host program would have access to standard OpenCL application programming interfaces (APIs), which allow data to be transferred to the FPGA, invoking the kernel on the FPGA and transferring the resulting data back.

Benefits for design engineers

One of the greatest advantages of OpenCL is portability. To overcome the increasing complexity, OpenCL has been developed to integrate heterogeneous systems and provide a single design environment platform-agnostic standard. This could provide significant productivity gains for engineers developing low-power, high-performance systems across high-performance computing, radar,

This technology allows engineers to achieve significantly faster time-to-market than the traditional FPGA design flow. An engineer can generate FPGA implementation of an OpenCL C code in a single step

video-processing, medical and other domains.

Utilising OpenCL also enables engineers to accelerate to systems while staying within a 'C' base environment, rather than lower-level hardware description language (HDL). It also allows greater design reuse by abstracting away low-level details. Being an open and royalty-free standard, there is a significant reduction in the cost of

development as well.

This technology allows engineers to achieve significantly faster time-to-market than the traditional FPGA design flow. An engineer can generate FPGA implementation of an OpenCL C code in a single step, bypassing the manual timing closure efforts and implementation of communication interfaces between the FPGA, host and external memories.

Moreover, this technology allows designers to describe their algorithms using the OpenCL C (based on ANSI C) parallel programming language instead of the traditional low-level HDL. What's more, it can be considered futureproof since you can re-target

your OpenCL C code to current and future FPGAs.

Promising outlook

OpenCL on FPGAs is just in its infancy but it has a lot of promise. There are areas where significant improvements can be made and fine tuning the OpenCL specification can make FPGAs even more useful.

This is a win-win approach, both for FPGA vendors and designers, as it will obviously generate more FPGA sales as well as provide developers with faster computing platforms. It also simplifies the methodology of getting logic into FPGAs since OpenCL is not RTL but 'C' code. This means software developers will be able to take advantage of FPGAs without having to use the current crop of tools. ●

The author is a tech correspondent at EY Bengaluru. The article is based on inputs from Altera's webcast and whitepaper on OpenCL program for FPGAs

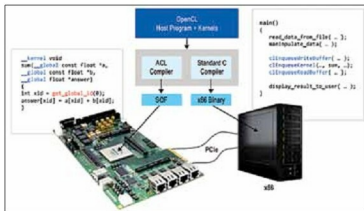


Fig. 3: OpenCL overview

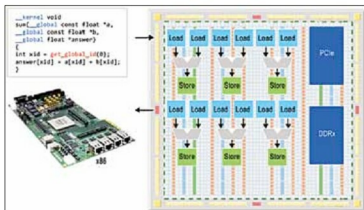


Fig. 4: Example of OpenCL implementation on an FPGA